

퀄리저널 개발 파일 사전

QualiJournal(퀄리저널) 개발 프로젝트의 주요 파일들을 Zip 패키지 구조(A.zip, B.zip)에 따라 정리한 자료입니다. 각 파일의 **경로**, **목적(역할)**, **관련 API 엔드포인트** 및 **사용 주체**, **실행 환경** 등을 표로 정리했습니다. 이를 통해 개발자는 어떤 파일이 어떤 역할을 하며 어디에서 실행되고, 어떤 API 기능에 영향을 주는지 빠르게 확인할 수 있습니다. 섹션별로 A.zip(관리자 시스템 코드)과 B.zip(자동 수집 파이프라인 코드)으로 구분했고, 필요한 경우 파일 내 주요 기능도 함께 요약했습니다.

A.zip - QualiJournal 관리자 시스템 코드 구조

A.zip에는 QualiJournal의 관리자 대시보드 웹 UI와 백엔드 API 서버 코드가 포함되어 있습니다. FastAPI 기반의 서버(`server_quali.py`)와 단일 HTML로 구현된 관리자 UI(`index.html`), 구성 설정(`config.json`), 기사 데이터(JSON), 유틸리티 스크립트 등이 한 프로젝트로 구성되어 있습니다 ¹ ². Cloud Run 환경에서 동작하도록 Docker 설정과 CI/CD 파이프라인 설정도 포함되어 있습니다. 아래 표들은 A.zip의 폴더 구조별로 파일들을 분류하여 상세 정보를 제공합니다.

핵심 애플리케이션 파일 (프로젝트 루트)

파일 경로	목적	관련 API 엔드포인트	사용 주체
<code>server_quali.py</code>	FastAPI 기반 API 서버 구현 파일. 관리자용 백엔드 엔드포인트를 정의하고, 기사 데이터(JSON) 및 각종 스크립트를 호출하여 보고서 생성 , 뉴스 요약/번역 , 내보내기(Export) , 시스템 상태 조회 등을 처리합니다 ¹ . 서버 실행 시 <code>config.json</code> 설정과 환경변수를 로드하고, 필요에 따라 <code>archive/</code> 나 <code>data/</code> 폴더의 JSON을 캐싱합니다 ¹ ³ . 토큰 인증과 CORS 설정이 적용되어 있으며, 관리자 UI에 정적 파일(<code>index.html</code>)도 제공하도록 설정됩니다.	<code>/api/report</code> (데일리 리포트 생성) ⁴ , <code>/api/enrich/keyword</code> 및 <code>/api/enrich/selection</code> (뉴스 요약/번역) ⁵ ⁶ , <code>/api/export/md</code> , <code>/api/export/csv</code> (Markdown/CSV로 최종본 다운로드) ⁷ ⁸ , <code>/api/status</code> (시스템 현황/KPI 조회) ⁹ , <code>/api/config/*</code> (설정값 조회/변경. 예: <code>/api/config/gate_required</code> 게이트 임계치 업데이트) ¹⁰ 등 모든 관리자용 API 를 핸들링합니다.	관리자 UI의 JS fetch 호출, CLI 테스트 (pytest) 등. 외부에서는 인증된 관리자만 사용.

파일 경로	목적	관련 API 엔드포인트	사용 주체
index.html	관리자 대시보드 UI 프론트엔드. 단일 HTML 페이지로, 내장된 JavaScript를 통해 백엔드 API를 호출하고 결과를 표시하는 SPA 형태입니다. 전체 수집 기사 수, 승인 기사 수 등의 KPI 지표를 보여주며 “일일 보고서 생성”, “키워드 요약/번역”, “선정보 요약/번역”, “Export MD”, “Export CSV” 등의 작업 버튼을 제공합니다 ². 사용자가 버튼을 클릭하면 JS에서 대응되는 <code>/api/...</code> 엔드포인트를 호출하여 작업을 수행하고, 응답에 따라 결과 링크 표시 또는 파일 다운로드를 진행합니다 ^{2 11}. 또한 편집자가 기사 목록에서 승인 체크박스를 선택하거나 코멘트를 입력하면 해당 변경사항을 서버로 보내 JSON 데이터를 업데이트합니다 (예: 기사 승인/코멘트 저장 API). UI에서는 작업 진행 중 로딩 메시지, 성공 시 결과물 링크 표시, 실패 시 오류 알림 등을 처리합니다 ^{2 12}.	(UI 자체에는 엔드포인트 없음 - 백엔드 API를 호출) UI에서 호출하는 주요 API는 <code>/api/report</code>, <code>/api/enrich/keyword</code>, <code>/api/enrich/selection</code>, <code>/api/export/md</code>, <code>/api/export/csv</code>, <code>/api/status</code> 등이며, 기사 승인/코멘트 반영을 위한 API (예: <code>/api/article/update</code> 등)도 사용됩니다.	관리자 (웹 브라우저를 통해 사용).
config.json	시스템 전역 구성 설정 파일. 품질 게이트 임계값, 기능 토글, 커뮤니티 필터 설정, 외부 RSS 관련 설정값, 용어집/번역 등 AI 관련 설정 등이 JSON으로 저장되어 있습니다 ^{13 14}. 서버 구동 시 이 파일을 로드하여 설정을 적용하며, 일부 값(예: <code>features.require_editor_approval</code>, <code>features.gate_required</code> 최소 승인기사 수 등)을 참고합니다 ¹⁵. 또한 커뮤니티 소스 필터(예: 최소 추천수, 차단 키워드)나 <code>external_rss</code> 게이트 설정값 등이 포함되어 뉴스 수집 및 평가 로직에도 활용됩니다 ¹⁶. 관리자 API를 통해 설정을 변경할 경우(<code>/api/config/...</code>), 서버 메모리의 설정을 업데이트함과 동시에 이 파일을 갱신하여 변경 내용을 영구 저장합니다 ^{10 17}.	<code>/api/config/*</code> (ex: <code>/api/config/gate_required</code>로 게이트 임계값 변경 시 해당 설정 업데이트) ¹⁰. 또한 뉴스 수집 파이프라인에서 이 설정값들을 읽어 품질 게이트나 커뮤니티 필터 기준으로 사용합니다.	백엔드 서버 (초기화 시 로드, 변경 시 갱신) 및 수집 파이프라인(필요 시 설정 읽기). 관리자도 필요시 수동 편집 가능.
official_sources.json	공식 뉴스 소스 목록 파일. 신뢰할 만한 공식 뉴스 사이트들의 RSS 피드 URL과 분류(category) 정보 등을 담고 있습니다 ^{18 19}. 자동 수집 파이프라인은 이 리스트를 순회하며 최신 기사를 수집합니다. 새로운 공식 출처 추가나 변경 시 이 파일을 편집하여 반영합니다 ¹⁸. 운영 중에는 정기적으로 RSS 링크가 유효한지(404 여부 등) 모니터링하고, 필요하면 URL을 업데이트해야 합니다 ²⁰.	(엔드포인트 직접 제공 없음; 수집 스크립트에서 참조)	자동 수집 스크립트(공식 기사 크롤러)에서 사용. 관리자가 수동으로 업데이트.

파일 경로	목적	관련 API 엔드포인트	사용 주체	설명
community_sources.json	커뮤니티 소스 목록 및 필터 설정 파일. Reddit 등 커뮤니티 출처의 API 목록과 필터 기준(예: 키워드 필터, 최소 추천수, 허용 도메인, 차단 패턴 등)을 정의합니다 ²¹ ²⁰ . 자동 수집 시 이 설정에 따라 커뮤니티 글을 필터링하여 수집합니다 (예: 추천수와 댓글 수 기준으로 걸러냄 ²⁰). 새로운 커뮤니티 출처 추가나 필터 조정 시 이 파일을 편집합니다.	(엔드포인트 직접 제공 없음; 수집 스크립트 에서 참조)	자동 수집 스크립트(커뮤니티 크롤러)에서 사용. 관리자가 필요 시 업데이트.	로컬 파일
paper_sources.json	학술/문헌 소스 목록 파일. 업계 백서, 표준 문서, 논문 등의 RSS/API 목록을 위한 파일입니다 ²² . (현재 해당 기능은 초기 단계이므로 사용 여부는 선택적입니다. 필요 시에만 작성하여 사용). 이 파일에는 기술 논문이나 표준 문서 출처의 피드 URL과 분류를 담을 수 있으며, 수집기에서 활용할 수 있습니다.	(엔드포인트 직접 제공 없음; 수집 스크립트 에서 참조 가능)	자동 수집 스크립트(논문/표준 수집기)에서 잠재적으로 사용.	로컬 파일
requirements.txt	프로젝트의 Python 의존 패키지 목록 . FastAPI, uvicorn, feedparser, requests, pandas 등 QualiJournal 시스템 구동에 필요한 라이브러리들이 명시되어 있습니다. Docker 이미지 빌드 시 이 파일을 참조하여 <code>pip install -r requirements.txt</code> 로 패키지를 설치합니다 ²³ ²⁴ . 개발 환경에서도 이 파일을 이용해 패키지를 세팅합니다.	(엔드포인트 없음)	개발자, CI (빌드 시), 배포 컨테이너 에서 사용.	C, R, D, A, C
Dockerfile	Cloud Run 배포용 도커 컨테이너 설정 파일 . 프로젝트의 컨테이너 이미지를 빌드하는 명세를 포함합니다. Python 기반 이미지 위에 소스 코드를 복사하고 requirements.txt 설치, 최종적으로 uvicorn으로 <code>server_quali:app</code> 을 8080 포트에서 실행하도록 CMD를 설정합니다 ²⁵ ²⁶ . (정적 파일 제공을 위해 Starlette의 <code>StaticFiles</code> 세팅이 포함될 수 있고, Cloud Run 기본 포트 8080 사용에 맞춰져 있습니다 ²⁴ .) 이 Dockerfile은 CI/CD 파이프라인에서 자동으로 빌드/배포에 활용되며, 운영 환경에서 컨테이너 설정 참고 자료이기도 합니다.	(엔드포인트 없음)	GitHub Actions (CI에서 이미지 빌드), Cloud Run (이미지 배포/실행). 개발자가 로컬에서 컨테이너 빌드 테스트 시 사용.	G, A, D, C, R, E
.env	개발/배포 환경변수 설정 파일. ADMIN_TOKEN, API_KEY, DB_URL, GCS_BUCKET 등 민감한 설정값을 키=값 형태로 보관합니다 ²⁷ ²⁸ . 이 파일은 Git에는 커밋하지 않으며 ²⁹ , 로컬 개발이나 CI에서 로드하여 환경변수를 세팅하는 용도로 사용됩니다. Cloud Run 배포 시에는 해당 값들을 콘솔 환경변수에 직접 설정하고 <code>.env</code> 내용은 로컬/CI 참고용으로 유지합니다 ²⁸ .	(엔드포인트 없음)	개발자(로컬 실행 시 <code>dotenv</code> 로 로드), CI(배포 전 환경 세팅), Cloud Run (배포 시 환경변수로 주입).	로컬, G, A, C, R, E

파일 경로	목적	관련 API 엔드포인트	사용 주체
<code>.github/workflows/ci.yml</code>	GitHub Actions 기반 CI/CD 파이프라인 설정 파일. 메인 브랜치에 푸시될 때 자동으로 Docker 이미지를 빌드하고 GCP Artifact Registry에 푸시한 뒤 Cloud Run에 배포하는 Workflow가 정의되어 있습니다 ²⁹ . 또한 테스트 실행, 린트 등 품질 체크도 이 YAML에 포함될 수 있습니다. 이 작업을 통해 배포 자동화 가 이루어지며, commit SHA 태그, 트래픽 라우팅 등의 Cloud Run 배포 옵션도 설정됩니다 ³⁰ ³¹ .	(엔드포인트 없음)	GitHub Actions (소스 푸시 시 워크플로우 실행), 개발자 (CI 파이프라인 모니터링).

데이터 파일 (data/ 폴더)

파일 경로	목적	관련 API 엔드포인트	사용 주체
<code>data/ selected_keyword_articles.json</code>	해당 날짜 수집된 전체 기사 목록 데이터 (키워드 기준). 하루 키워드에 대해 자동 수집된 모든 기사들의 정보가 배열로 저장됩니다 ³² ³³ . 각 기사 객체에는 제목(title), URL, 영문 요약문(summary_en), 출처(source)와 날짜, 점수(score), 승인 여부(approved), 편집자 코멘트(editor_note) 등의 필드가 포함됩니다 ³³ . 이 파일은 자동 수집 단계에서 생성/갱신 되며, 편집자는 관리자 UI를 통해 각 기사별 <code>approved</code> 를 <code>true</code> 로 바꾸고 <code>editor_note</code> 를 추가하는 방식으로 이 데이터를 편집합니다 ³⁴ . JSON 형식이므로 필요하다면 에디터로 직접 열어 수정할 수도 있지만, Admin UI 를 통해 조작하는 것이 안전합니다 ³⁴ .	<code>/api/status</code> (총 수집 기사 수 등 조회 시 이 파일 기반 집계) ³⁵ ; <code>/api/enrich/keyword</code> (모든 수집기사 요약 요청 시 이 파일을 입력으로 사용) ³⁶ ; Admin UI 대시보드 (기사 테이블 표시 및 승인/코멘트 편집).	백엔드 서버 (요약/보고서 생성 시 읽기), 관리자 UI (리스트 표시 및 수정), 자동 수집 스크립트 (작성/갱신).

파일 경로	목적	관련 API 엔드포인트	사용 주체
<code>data/selected_articles.json</code>	최종 선정된 기사 목록 데이터. <code>selected_keyword_articles.json</code> 중 편집자가 승인(<code>approved=true</code>)한 기사만 모아놓은 JSON 파일입니다 ³² ³⁴. 발행 대상이 되는 최종본으로, 일일 보고서 생성이나 Export시 이 파일의 목록을 활용합니다. 보통 Admin UI에서 편집자가 기사 승인/취소를 하면 백엔드가 이 파일을 갱신하거나 별도로 동기화합니다. (예: UI에서 승인 작업 완료 시 서버가 자동으로 승인된 기사만 추려 이 파일을 저장하거나, 필요시 <code>tools/sync_selected_for_publish.py</code>로 갱신 ³⁷.) 최종 발행 후에는 이 파일 내용을 <code>archive/</code> 폴더에 날짜별 JSON으로 보관 (아카이브)할 수 있습니다 ³⁸.	<code>/api/report</code> (보고서 생성 시 승인 기사 목록으로 활용) ⁴; <code>/api/export/md</code> & <code>/api/export/csv</code> (최종본 Markdown/CSV 추출 시 이 파일 데이터 사용) ⁷ ⁸; <code>/api/enrich/selection</code> (승인본만 요약 요청 시 입력 데이터로 사용) ⁶; Admin UI (발행 준비된 기사만 필터링해서 표시 가능).	백엔드 서버 (보고서/Export 생성 시 읽기·쓰기), 관리자 UI(승인된 기사 리스트 표시), 발행 스크립트/기능 (아카이브 저장).

유틸리티 스크립트 (tools/ 폴더)

파일 경로	목적	관련 API 엔드포인트
<code>tools/make_daily_report.py</code>	일일 보고서 Markdown 생성 스크립트. 현재까지 승인된 기사 목록 (<code>selected_articles.json</code>)을 모두 모아 당일 데일리 리포트를 Markdown 형식으로 생성합니다 ³⁹. 출력 파일은 <code>archive/reports/YYYY-MM-DD_report.md</code> 형태로 저장되며 (키워드가 있으면 파일명에 포함) ⁴, 생성 과정에서 오류가 발생해도 기존 데이터로 최소한의 보고서를 만들어내도록 설계되어 항상 결과를 얻을 수 있습니다 ⁴ ⁴⁰.	POST <code>/api/report</code> - 호출 시 백이 스크립트를 실행하여 보고서를 생성 완료 후 API 응답으로 보고서 파일 경로를 반환합니다 (예: <code>{ "ok": true, "path": "archive/reports/2025-10-26_report.md" }</code>).

파일 경로	목적	관련 API 엔드포인트
<code>tools/enrich_cards.py</code>	뉴스 요약 및 번역 스크립트. 수집된 기사들에 대한 영어 요약문을 생성하고, 필요시 한국어 번역을 수행하여 JSON 데이터 필드에 반영해주는 도구입니다 ⁴¹. 외부 번역 API 키(<code>API_KEY</code>)가 설정된 경우 해당 API를 통해 2단계 요약·번역을 하고, 키가 없으면 규칙 기반 요약으로 대체합니다 (OpenAI 등 사용 가능). 키워드 전체 기사에 대해 수행하거나 승인된 선정본 기사만 대상으로 수행할 수 있습니다. 실행 결과 각 기사 객체에 <code>summary</code>나 <code>summary_ko</code> 필드가 채워지고, 전체 기사 요약본을 Markdown 파일로도 저장합니다 (예: <code>archive/enriched/YYYY-MM-DD_ALL.md</code> 혹은 <code>_SELECTED.md</code>) ⁴².	POST <code>/api/enrich/keyword</code> - 키워드의 전체 기사 요약/번역 수행 (이 결과를 호출하여 <code>selected_keyword_articles</code> 모든 기사에 대해 요약/번역) ³⁶. POST <code>/api/enrich/selection</code> - 선정본 기사들만 요약/번역 수행 (스크립트 호출된 기사만 처리) ⁶. 각 API는 완료 결과 Markdown 파일 경로를 JSON으로 합니다. 실패 시 오류 메시지를 포함하며, UI에서 이를 표시합니다 ⁴³.
<code>tools/rebuild_kw_from_today.py</code>	원본 데이터 병합 스크립트. 자동 수집된 여러 원본 JSON들을 하나의 표준 구조로 병합하여 <code>selected_keyword_articles.json</code>을 생성/갱신합니다 ⁴⁴. 예를 들어 공식 소스별/커뮤니티별 개별 JSON 결과물이 있다면 이들을 합쳐서 하루 키워드 기준의 통합 기사 목록을 만듭니다. 수집 파이프라인 수행 중 또는 수집 후에 이 스크립트를 실행하여 당일자 기사 목록을 정리합니다.	(직접 엔드포인트 없음)
<code>tools/augment_from_official_pool.py</code>	공식 뉴스 풀 보강 스크립트. 승인된 기사 수가 부족할 때를 대비하여, 사전에 정의된 공식 뉴스 풀에서 추가 기사를 가져와 보충합니다 ⁴⁵. 예를 들어 당일 승인 기사가 너무 적으면, <code>official_sources</code>의 다른 키워드 뉴스나 이전 기사를 가져와 채워주는 용도로 사용 가능합니다. 운영자가 발행 품질을 맞추기 위해 수동으로 실행할 수 있는 보조도구입니다.	(엔드포인트 없음)
<code>tools/force_approve_top20.py</code>	일괄 승인 도구. 기사 점수 상위 20개를 자동으로 <code>approved=true</code>로 설정하는 스크립트입니다 ⁴⁶. 긴급하게 일정 수 이상의 기사를 신속히 승인해야 할 경우 활용합니다 (예: 편집 시간 부족 시). 이 스크립트를 실행하면 <code>selected_keyword_articles.json</code> 내 점수 필드 기준 상위 20개 기사에 <code>approved</code> 플래그를 True로 일괄 설정합니다.	(엔드포인트 없음)

파일 경로	목적	관련 API 엔드포인트
tools/ repair_selection_files.py	선택 JSON 구조 복구 스크립트. selected_keyword_articles.json 또는 selected_articles.json의 포맷이나 내용에 문제가 있을 경우 JSON 구조를 교정합니다 ³⁷. (예: 수동 편집 중 JSON 문법 오류 발생 시 이를 고쳐주는 기능). 잘못된 구조로 인해 서버나 보고서 생성에 오류가 날 때 이 도구를 사용해 문제를 해결합니다 ³⁷.	(엔드포인트 없음)
tools/ sync_selected_for_publish.py	최종선택 동기화 스크립트. selected_keyword_articles.json에서 approved=true인 항목들을 추려 selected_articles.json을 재생성합니다 ³⁷. 만약 JSON을 수동 편집했거나 승인 데이터가 일치하지 않을 경우 이 스크립트를 실행하여 최종 발행 대상 파일을 동기화할 수 있습니다. 즉, 편집된 승인 리스트를 다시 한 번 공식 최종본으로 반영하는 도구입니다.	(엔드포인트 없음)

테스트 스크립트 (tests/ 폴더)

파일/경로	목적	관련 API 엔드포인트	사용 주체	실행 위치
tests/ (디렉토리)	통합 테스트 코드 모음. Pytest 기반으로 Admin 시스템의 주요 기능에 대한 자동화 테스트를 제공합니다. 예를 들어, 가상의 기사 데이터를 만들어 끝부터 끝(E2E) 시나리오를 검증하는 테스트 (20개 수집 -> 15개 승인 -> /api/report 호출 -> /api/export/csv 호출까지 파이프라인 검증) 등이 포함됩니다 ⁴⁷. 개별 테스트에서는 FastAPI의 TestClient를 이용해 서버를 띄우고, monkeypatch로 파일 경로나 함수를 대체하여 파일 생성 동작을 검증합니다 ⁴⁸ ⁴⁹. 주요 테스트 시나리오로 보고서 생성 (/api/report), 요약 (/api/enrich/*), Export (/api/export/*), 게이트 임계치 변경 (/api/config/gate_required) 등이 커버되며, 최근 코드 변경이 기능에 영향을 주지 않는지 회귀 테스트합니다 ⁵⁰ ⁵¹.	/api/report, /api/enrich/keyword, /api/enrich/selection, /api/export/md, /api/export/csv, /api/config/gate_required 등 (테스트 대상 API 전체).	개발자, CI 파이프라인 (자동 테스트 실행).	로컬 개발 (pytest), GitHub Actions 등 CI 환경.

아카이브 및 기타

- archive/ 폴더: 발행된 결과물을 날짜별로 저장하는 아카이브 디렉토리입니다. 예를 들어 archive/reports/에는 일일 보고서 Markdown/HTML 파일들이, archive/enriched/에는 뉴스 요약본

Markdown 파일들이 저장됩니다 ⁵² ⁴² . 또한 발행 당시의 JSON 데이터도 보관할 수 있습니다 (예: `archive/2025-10-26.json` 은 해당 날짜의 `selected_articles.json` 아카이브본). 이러한 구조를 통해 과거 발행본을 기록/추적할 수 있습니다 ³⁸ . 다만 날짜별 파일들이 쌓이면 관리가 힘들 수 있어, 일정 기간 후에는 월별 폴더로 정리하거나 DB로 이전하는 방안도 고려됩니다 ³⁸ . 현재 Archive는 **Cloud Run 컨테이너 내 로컬 디스크**에 저장되므로, 영구 보관을 위해 GCS 버킷(`GCS_BUCKET`) 등에 복사하는 운용도 가능하도록 설계되었습니다. Archive 폴더의 파일은 Admin UI에서 링크로 열어보거나, 필요시 수동으로 다운로드하여 사용합니다 ⁵³ .

- **그 외:** 프로젝트 루트에는 `.gitignore`, `README.md` 등이 존재할 수 있으나, 핵심 로직과 직접 관련되지는 않으므로 생략합니다. 데이터베이스를 사용하는 경우 `DB_URL` 설정에 따라 Postgres 등 연결할 수 있으나, 기본 구현은 JSON 파일로 대체하고 있습니다.

B.zip – QualiJournal 자동 수집 파이프라인 코드

B.zip에는 QualiJournal의 **뉴스 자동 수집 및 초기 가공 파이프라인** 코드가 포함되어 있습니다. 이는 관리자 시스템에 앞서 뉴스 데이터를 모으고 평가하는 백엔드 프로세스로, Cron 작업이나 수동 실행을 통해 주기적으로 **뉴스/커뮤니티 글을 수집**하고 기본 **품질 평가(QG)** 및 **스코어링**을 수행해 JSON 데이터를 생성합니다. B.zip의 코드는 A.zip의 Admin 시스템과는 별개로 실행되며, 수집된 데이터를 A의 `data/selected_keyword_articles.json`에 공급하는 역할을 합니다. (궁극적으로 Admin UI에서 데이터를 다루기 위해 사전 단계로 동작합니다.) 주요 파일 및 구성은 다음과 같습니다.

자동 수집 파이프라인 주요 파일

파일 경로	목적	관련 API 엔드 포인트	사용 주체
jjippa_news_scheduler.py (예상)	뉴스 수집 스케줄러/파이프라인 메인 스크립트. QualiJournal의 자동화된 수집·평가·발행 파이프라인을 구현한 Python 스크립트입니다. (파일명은 예시이며, 개발 코드상 실제 이름) 이 스크립트는 공식 뉴스 소스(<code>official_sources.json</code>)와 커뮤니티 소스(<code>community_sources.json</code>)에 정의된 목록을 기반으로 각 출처의 최신 기사를 크롤링합니다. 수집된 기사들에 대해 품질 게이트(QG) 검사를 수행하여 본문 길이, 문장 수, 링크 수 등이 기준에 미달하는 기사는 걸러 내거나 상태 플래그(PASS/HOLD)를 지정합니다. 또한 각 기사에 점수(score)를 계산하여 중요도를 평가하고, (설정에 따라) 자동으로 상위 기사들을 선별할 수 있습니다. 커뮤니티 글의 경우 Reddit API 등을 통해 가져오며, 추천수·댓글수 및 키워드 매칭 등을 고려해 필터링합니다 ²⁰ ⁵⁴. 이렇게 수집/평가된 기사들은 내부적으로 JSON 구조로 모아지며, 결과를 <code>selected_keyword_articles.json</code> 형태로 저장하거나 갱신합니다. (필요한 경우 임시 raw JSON들을 생성한 뒤 <code>rebuild_kw_from_today.py</code>로 합쳐 최종 저장). 스크립트 실행 옵션에 따라 자동 발행까지 수행할 수도 있습니다. 예를 들어 모든 절차를 자동화하면, 승인 단계를 생략하거나 기본값으로 처리하여 곧바로 HTML/MD 보고서를 생성하고 <code>archive/</code> 폴더에 <code>YYYY-MM-DD.html/.md/.json</code>을 저장하는 기능도 갖추고 있습니다 ⁵⁵ ⁵⁶. 다만 현재 운영에서는 편집자 승인 단계를 위해 자동 발행은 사용하지 않고, 수집 및 평가까지만 이 스크립트를 활용합니다 (발행은 관리자 UI 통해 수행).	(별도의 HTTP API 엔드 포인트 없음 - 독립 실행형 스크립트로 동작)	자동 스케줄러(예: Cron job, Windows 작업 스케줄러 등)를 통한 동 실행, 또는 운영자가 필요 시 수동으로 실행.

파일 경로	목적	관련 API 엔드 포인트	사용 주체
(공동 사용) <code>official_sources.json</code>
 (공동 사용) <code>community_sources.json</code>
 (공동 사용) <code>paper_sources.json</code>	소스 리스트 및 설정 파일 - 해당 파일들은 A.zip에 포함되어 있으나, B.zip의 수집 스크립트에서도 동일하게 활용됩니다. 파이프라인 스크립트는 A의 <code>official_sources.json</code>, <code>community_sources.json</code> 등을 읽어 수집 대상과 필터 기준을 파악합니다. 예를 들어 공식 소스 리스트의 RSS URL을 순회하며 기사 HTML/피드를 가져오고, 커뮤니티 소스와 필터 설정을 참고하여 Reddit 등의 글을 가져온 뒤 추천수/댓글 기준으로 거릅니다 ¹⁸ ²⁰. <code>paper_sources.json</code>은 현재 적극 사용되지는 않지만, 향후 학술자료 수집 시 참조할 수 있습니다. 이러한 구성 파일들은 수집 파이프라인과 관리자 시스템이 공유하여 사용하므로, 업데이트 시 일관성이 유지되도록 합니다 ⁵⁸.	(엔드 포인트 없음; 파이프라인 내부 로직 사용)	자동 수집 스크립트(위 <code>jjippa_news_scheduler.py</code>서 로드), 운영자(파일 내용 업데이트)
(공동 사용) <code>config.json</code>	설정값 공유 파일 - 관리자 시스템의 설정 파일이지만, 수집 파이프라인 역시 필요한 경우 이를 참조합니다. 예를 들어 <code>features.gate_required</code>나 <code>community_filters</code> 등의 값은 파이프라인 동작(예: 자동 승인 최소 필요기사 수나 커뮤니티 필터 임계값)에 영향을 줄 수 있습니다 ¹³ ¹⁶. 파이프라인이 시작될 때 이 파일을 로드하여 기준값을 가져오도록 구현할 수 있습니다. (원본 File2 코드에서는 별도의 <code>editor_rules.json</code> 등을 사용했을 수 있으나, 현재는 config로 통합 관리) ⁵⁸.	(엔드 포인트 없음)	자동 수집 스크립트(동작 시 설정값 로드), 운영자(필요시 값 조정).

파일 경로	목적	관련 API 엔드 포인트	사용 주체
산출 및 기타	파이프라인 스크립트 실행 결과로 중간/최종 산출물이 생성됩니다. 주요 산출은 앞서 설명한 <code>data/selected_keyword_articles.json</code> (자동 생성)이며, 이 외에 스크립트가 완전 자동 발행 모드로 동작할 경우 <code>archive/</code> 폴더 하위에 당일의 HTML, Markdown, JSON 파일들을 직접 생성할 수도 있습니다 ⁵⁶. 하지만 현재 운영에서는 자동 발행을 사용하지 않으므로 이러한 파일은 관리자 UI의 수동 발행 절차에서 생성됩니다. 또한 파이프라인에서 로그를 통해 수집/필터링 결과를 출력하며, 실패 시 에러를 콘솔이나 Slack/Email로 알리는 기능도 구성할 수 있습니다 ⁵⁷ (현재 기본 로깅만).	(해당 없음)	운영자(결과물 모니터링), 관리자(필시 산출물 활용).

주의: 자동 수집 파이프라인(B.zip)은 관리자 시스템(A.zip)과 분리되어 동작하지만, **동일한 파일들을 공유**하여 사용하고 있습니다. 따라서 코드 변경이나 구성 수정 시 양쪽에 미치는 영향을 고려해야 합니다. 예를 들어 소스 목록을 갱신하면 수집기와 UI 모두에 반영되고, 품질 게이트 수치를 바꾸면 (`config.json` 수정 또는 API 통해) 다음 수집 실행에 적용됩니다. 현재 QualiJournal 운영에서는, **매일 일정 시각** 이 파이프라인 스크립트를 실행하여 당일의 기사 목록을 확보한 후 편집자는 관리자 UI에서 승인/편집을 진행하고, 이후 필요한 보고서/발행을 수행하는 형태로 이루어집니다. 이처럼 두 시스템이 연계되어 QualiJournal의 전체 기능을 구성합니다.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

29

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

1011_3QualiJournal 관리자 시스템

최종 가이드북.pdf

file://file-6Hu18vVrjQweSb5SZz6a6M

27

28

1014_2QualiJournal 관리자 시스템 가이드.pdf

file://file-V1gqDLDq7V1uefyJAegEJS

30

31

1022_1Cloud Run 배포 확인 및 최신 리버전 트래픽 적용 가이드.pdf

file://file-7X7JDAg5PPzVgjeeR8DG8V

54

56

58

1003_3A 하루 한 키워드” 역사 큐레이션형 퀄리뉴스를 구축하기 위한 구체적 개발 가이드입니다.pdf

file://file-92Q3asQNjsGDWWy73Fkbpt

55

1003_1A퀄리 뉴스 제안.pdf

file://file-9ckyN6UsWeG3crtkDBreya

57

1004_3A작업계획report.pdf

file://file-S3G4TQFPei3GeAJcXjN6B3